Graphic Mode

เครื่องมือที่ใช้ : Processing4.0a4

ข้อที่ 1 ทดลองวาดรูปและสร้างแอนิเมชันโดยใช้ตัวอย่างจาก lecture slides และสังเกตผลที่ได้

จากสไลด์หน้า 45-46 จะได้โค้ดตามตัวอย่างด้านล่าง

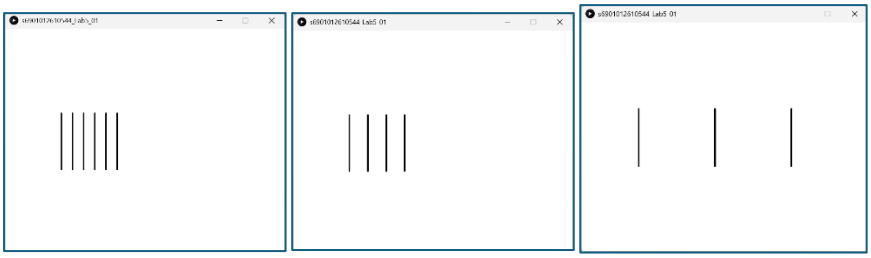
```
def setup():
    size(500, 400)
    stroke(0)
    strokeWeight(3)
    noLoop()

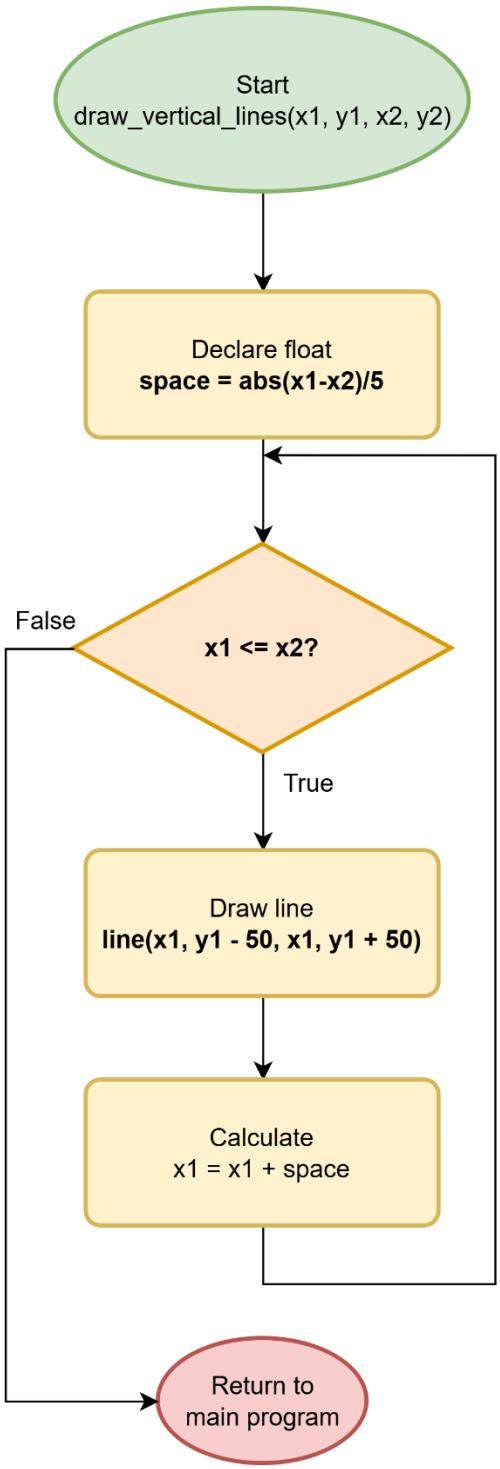
def draw():
    background(255)
    draw_vertical_lines(100, 200, 200, 200)

def draw_vertical_lines(x1, y1, x2, y2):
    space = abs(x1 - x2)/5

    while x1 <= x2:
        line(x1, y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```

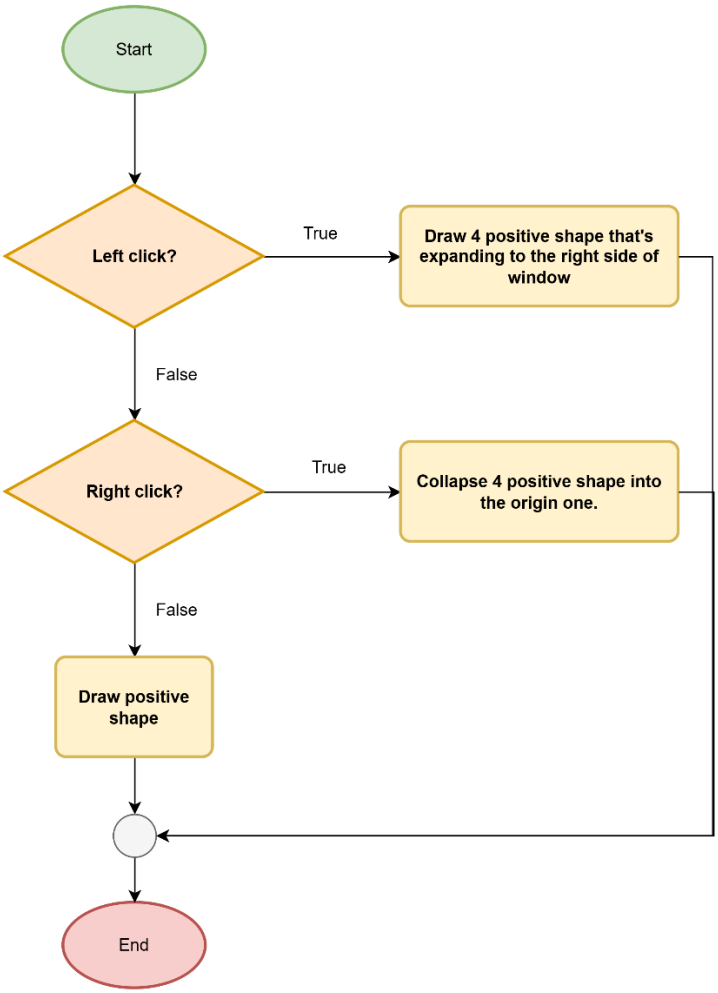
โดยการสร้างฟังก์ชัน $draw_vertical_ines(x1, y1, x2, y2)$ จะเป็นการสร้างเส้นตรงในแนวตั้งขึ้นมา ทั้งหมด 6 เส้น (สามารถปรับจำนวนตรงผลหารของตัวแปร space จะได้เส้นออกมาก n + 1 เส้น) และสามารถปรับให้รูปแคบกว้างได้จากพารามิเตอร์ทั้งหมด

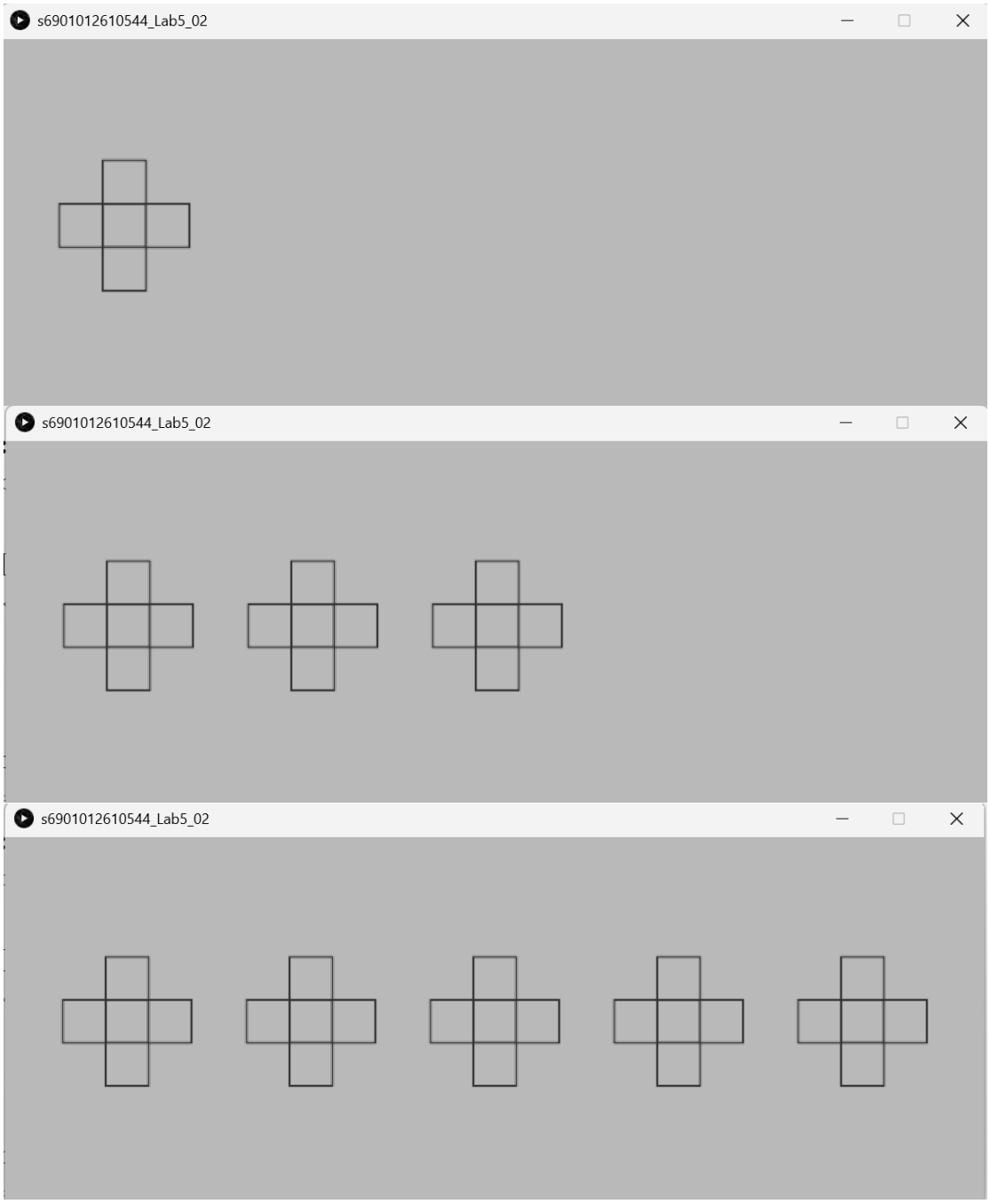


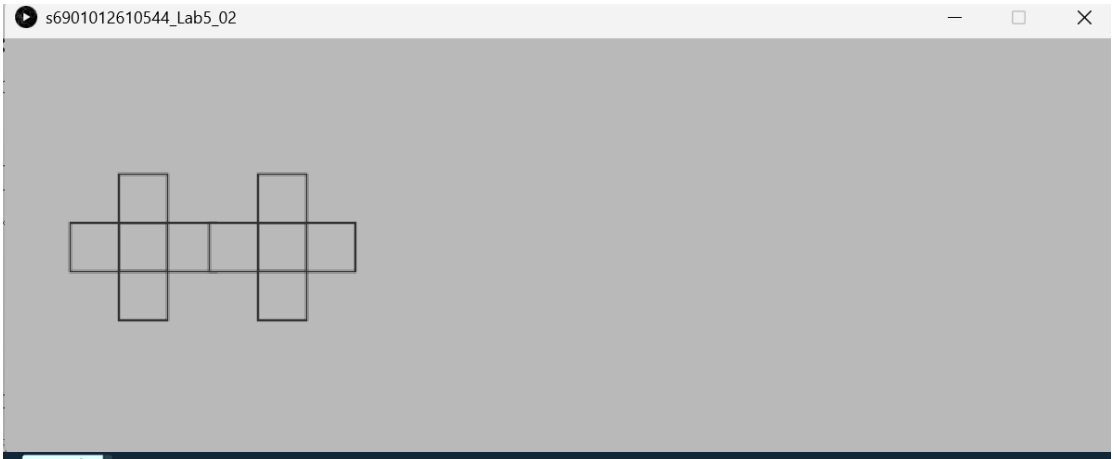

Start
draw_vertical_lines(x1, y1, x2, y2)
Declare float
space = abs(x1-x2)/5
x1 <= x2?
False
True
Draw line
line(x1, y1 - 50, x1, y1 + 50)
Calculate
x1 = x1 + space
Return to
main program

ข้อที่ 2 ใช้draw_positive() จาก lab 3, เพื่อวาดรูปคล้ายด้านล่าง โดยใช้คำสั่งวนซ้ำเพิ่ม Animation และ Interaction ต่างๆ

จาก draw_positive() ใน lab 3 ได้มีการเพิ่มทั้ง Animation และ Interaction โดยเริ่มแรกจะมีเครื่องหมายบวกเพียงรูปเดียว เมื่อคลิกซ้ายจะมีรูปบวกที่เหลือสไลด์ออกมาที่ละรูปจนครบ และเมื่อคลิกขวาก็จะสไลด์กลับเข้าไปยังที่เดิมที่ละรูปจนครบ




s6901012610544_Lab5_02
s6901012610544_Lab5_02
s6901012610544_Lab5_02




```
num_shapes = 5
base_x = 100.0
spacing = 150.0
y_pos = 150.0
square_size = 35.0
speed = 10.0

current_x = []
target_x = []

is_expanding = False
is_collapsing = False

def setup():
    size(800, 300)
    rectMode(CENTER)

    i = 0
    while i < num_shapes:
        current_x.append(base_x)
        target_x.append(base_x + (i * spacing))
        i += 1

def draw_positive(x, y, s):
    draw_square(x, y, s)
    draw_square(x, y - s, s)
    draw_square(x, y + s, s)
    draw_square(x - s, y, s)
    draw_square(x + s, y, s)

def draw():
    global is_expanding, is_collapsing
    background(185)

    if is_expanding:
        i = 1
        while i < num_shapes:
            if current_x[i] < target_x[i]:
                current_x[i] += speed
                if current_x[i] > target_x[i]:
                    current_x[i] = target_x[i]
                break
            i += 1

        if current_x[num_shapes - 1] >= target_x[num_shapes - 1]:
            is_expanding = False
```

```
    elif is_collapsing:
        i = num_shapes - 1
        while i >= 1:
            if current_x[i] > base_x:
                current_x[i] -= speed
                if current_x[i] < base_x:
                    current_x[i] = base_x
                break
            i -= 1

        if current_x[1] <= base_x:
            is_collapsing = False

    fill(185)
    stroke(50)
    strokeWeight(1.5)

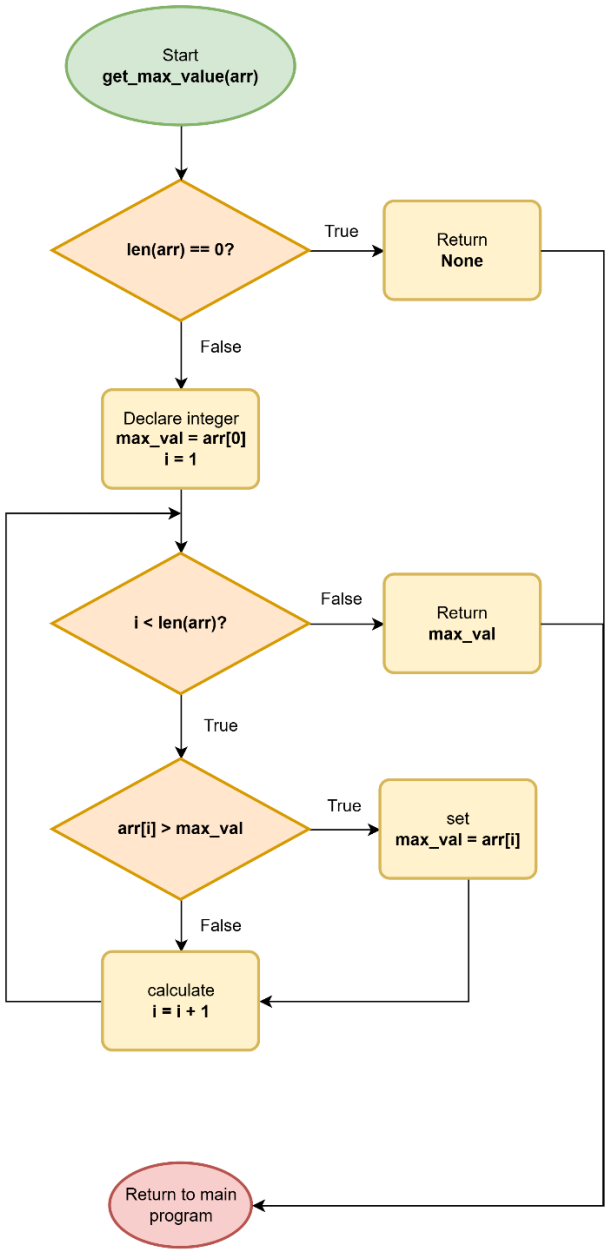
    i = 0
    while i < num_shapes:
        if i == 0 or current_x[i] > base_x:
            draw_positive(current_x[i], y_pos, square_size)
        i += 1

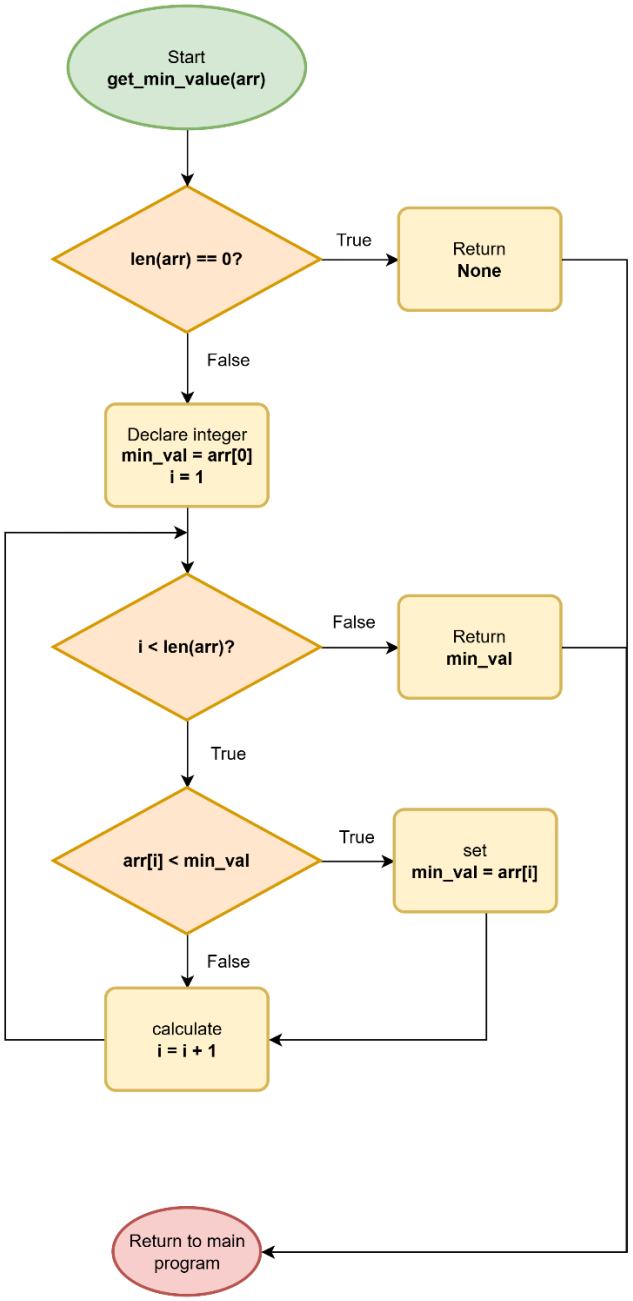
def mousePressed():
    global is_expanding, is_collapsing
    if mouseButton == LEFT:
        is_expanding = True
        is_collapsing = False
    elif mouseButton == RIGHT:
        is_collapsing = True
        is_expanding = False
```

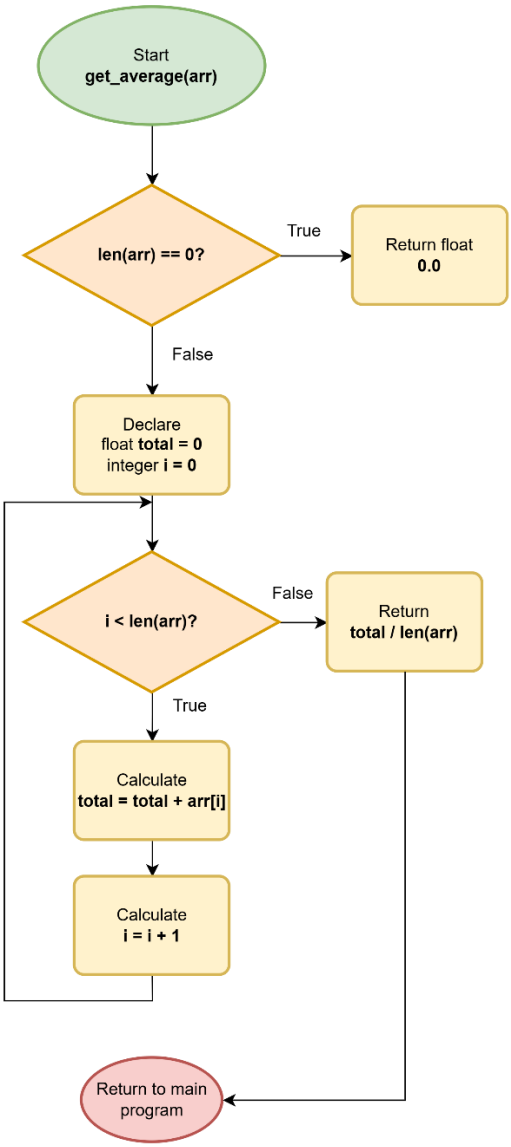
Text Mode

ข้อที่ 1 write functions to

a. find max/min/average value of an array




Start
get_min_value(arr)
len(arr) == 0?
True
Return
None
False
Declare integer
min_val = arr[0]
i = 1
i < len(arr)?
False
Return
min_val
True
arr[i] < min_val
True
set
min_val = arr[i]
False
calculate
i = i + 1
Return to main
program


Start
get_average(arr)
len(arr) == 0?
True
Return float
0.0
False
Declare
float total = 0
integer i = 0
i < len(arr)?
False
Return
total / len(arr)
True
Calculate
total = total + arr[i]
Calculate
i = i + 1
Return to main
program

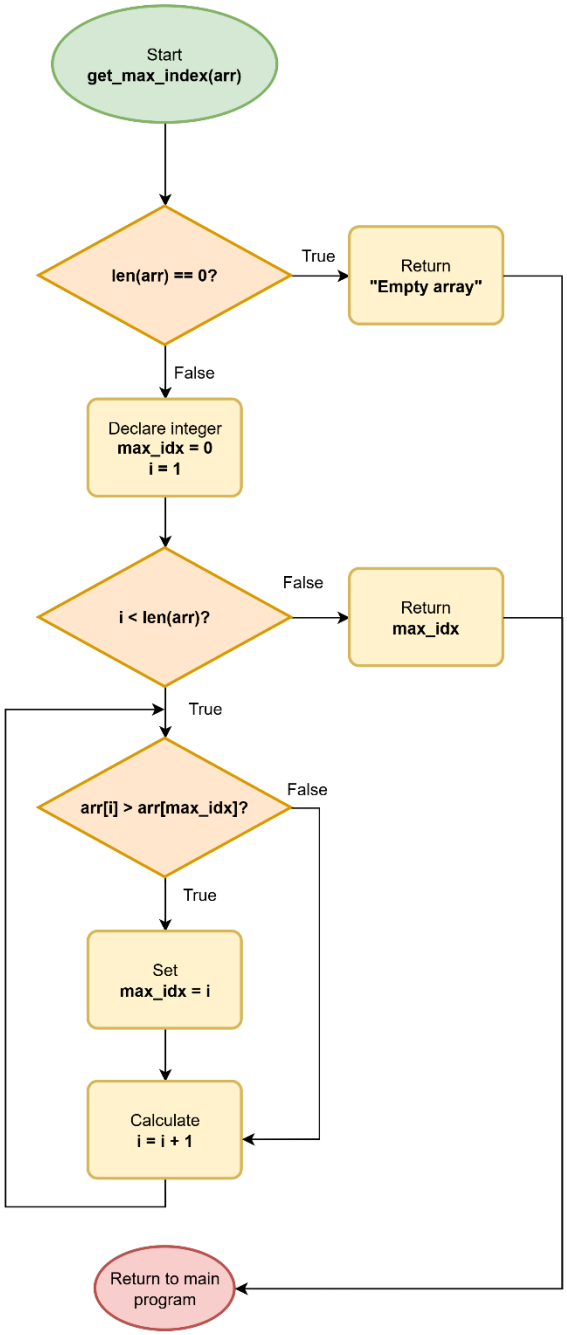
```python
#a. Max, min, and average value of an array
def get_max_value(arr):
    if len(arr) == 0:
        return None
    max_val = arr[0]
    i = 1
    while i < len(arr):
        if arr[i] > max_val:
            max_val = arr[i]
        i += 1
    return max_val

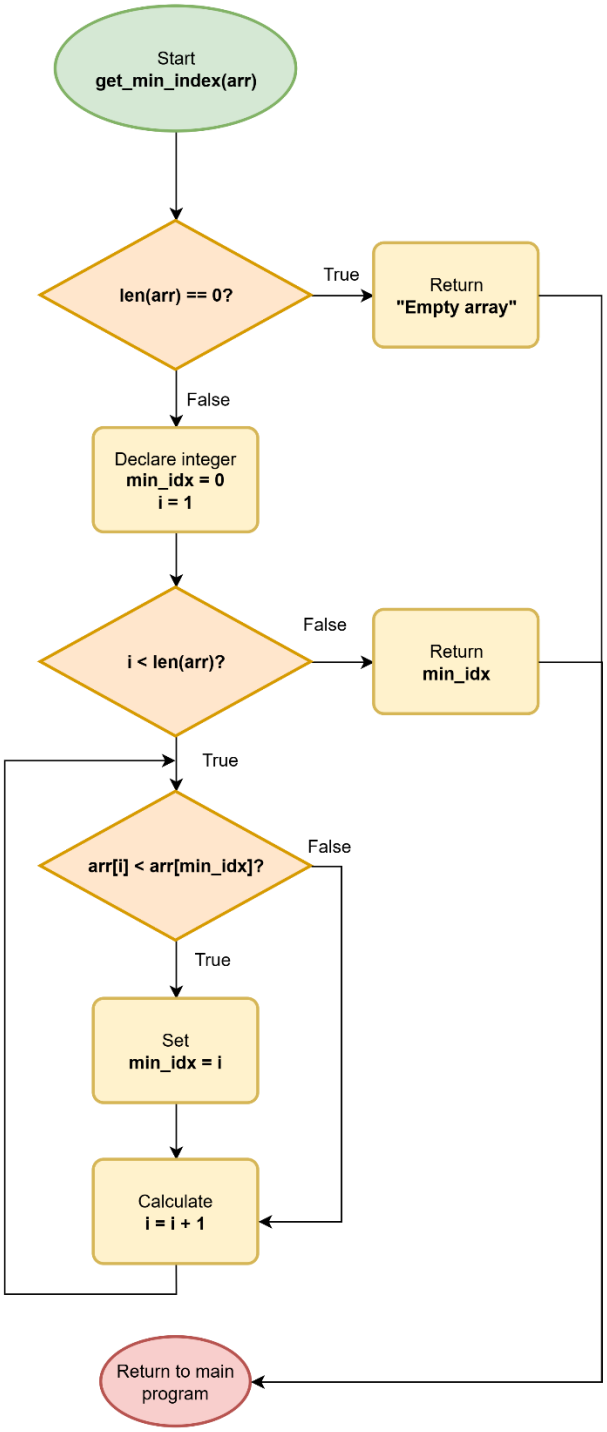

def get_min_value(arr):
    if len(arr) == 0:
        return None
    min_val = arr[0]
    i = 1
    while i < len(arr):
        if arr[i] < min_val:
            min_val = arr[i]
        i += 1
    return min_val


def get_average(arr):
    if len(arr) == 0:
        return 0.0
    total = 0
    i = 0
    while i < len(arr):
        total += arr[i]
        i += 1
    return total / len(arr)
```

```
Array: [3, 8, 1, 14, 9, 22, 6]
Max Value: 22
Min Value: 1
Average Value: 9.0
```

b. find index of max/min value of an array



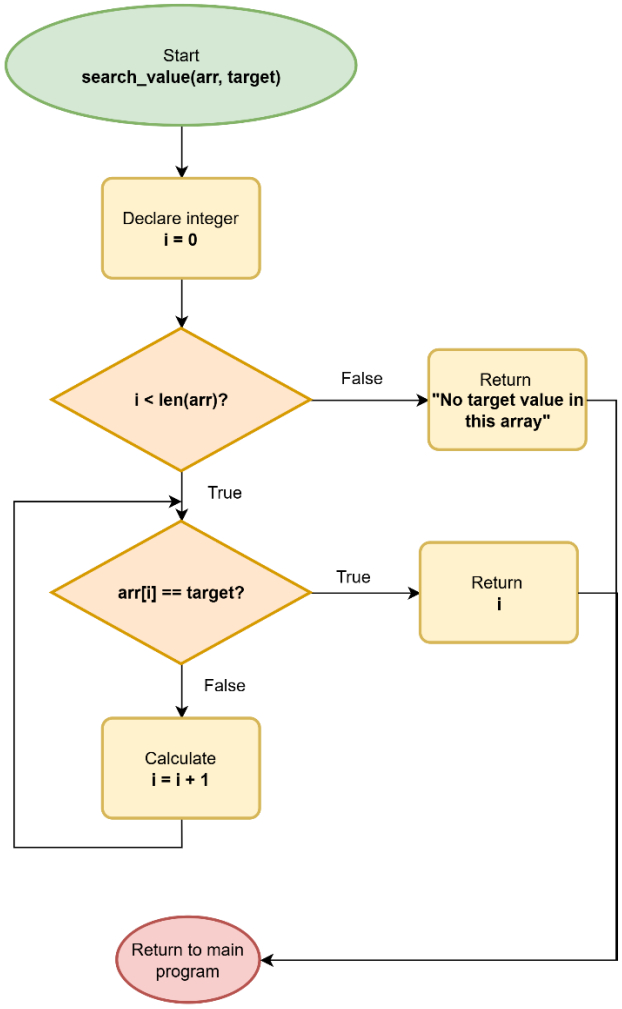

Start
get_min_index(arr)
len(arr) == 0?
True
Return
"Empty array"
False
Declare integer
min_idx = 0
i = 1
i < len(arr)?
False
Return
min_idx
True
arr[i] < arr[min_idx]?
False
True
Set
min_idx = i
Calculate
i = i + 1
Return to main
program

```python
#b. Index of max and min value of an array
def get_max_index(arr):
    if len(arr) == 0:
        return "Empty array."
    max_idx = 0
    i = 1
    while i < len(arr):
        if arr[i] > arr[max_idx]:
            max_idx = i
        i += 1
    return max_idx


def get_min_index(arr):
    if len(arr) == 0:
        return "Empty array"
    min_idx = 0
    i = 1
    while i < len(arr):
        if arr[i] < arr[min_idx]:
            min_idx = i
        i += 1
    return min_idx
```

```
Array: [3, 8, 1, 14, 9, 22, 6]
Index of Max: 5
Index of Min: 2
```

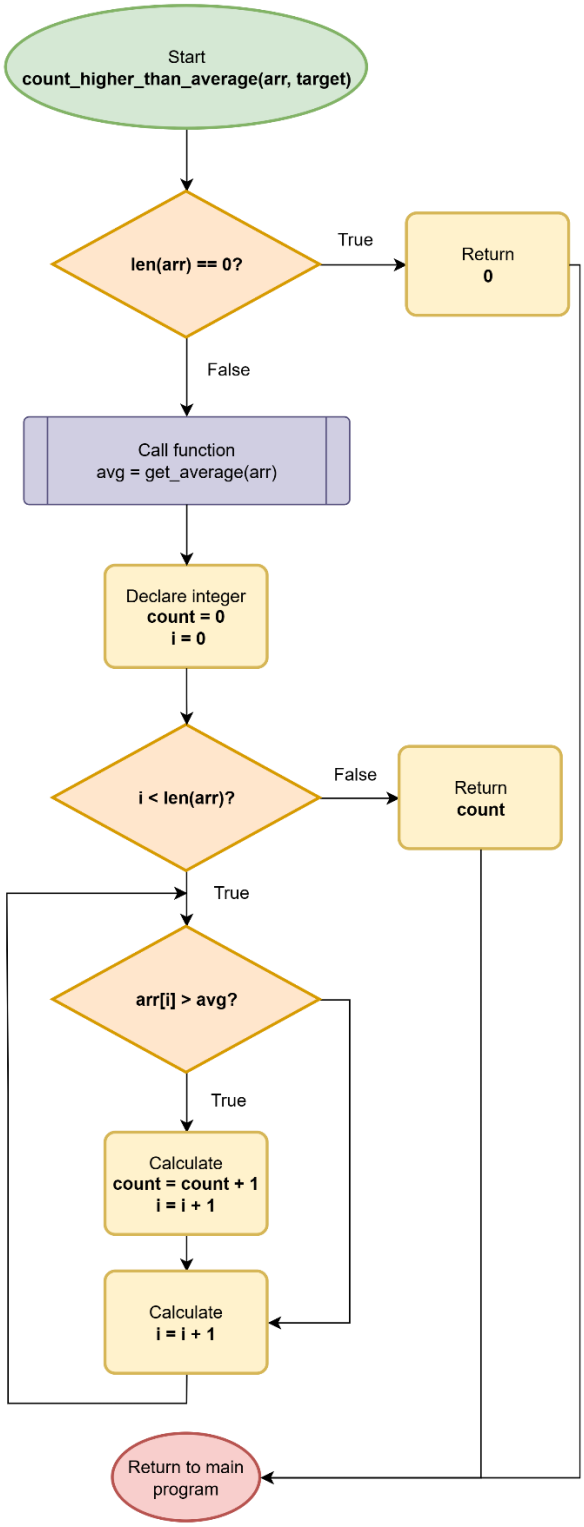
c. search for a value in an array (return index)




```python
#c. Search for a value in an array (returns first matching index or -1)
def search_value(arr, target):
    i = 0
    while i < len(arr):
        if arr[i] == target:
            return i
        i += 1
    return "No target value in this array"
```

```
Array: [3, 8, 1, 14, 9, 22, 6]    target_to_find = 14
Index of 14: 3
```

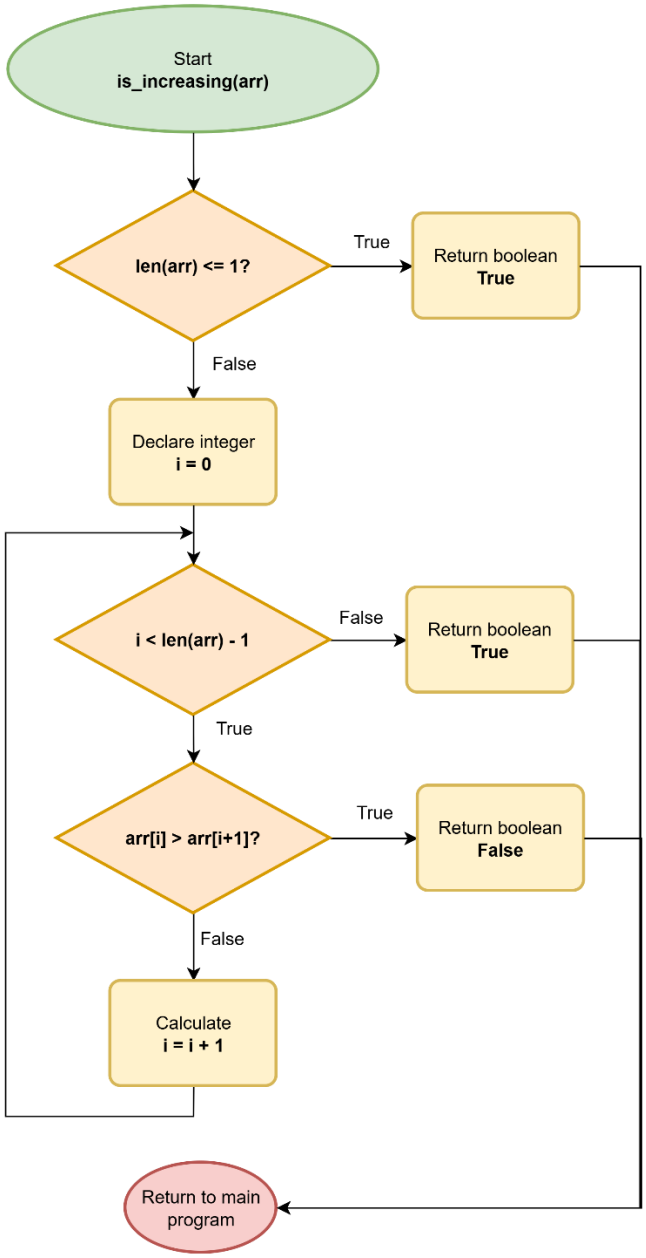
d. count number of elements that have value higher than average


Start
count_higher_than_average(arr, target)
len(arr) == 0?
True
Return
0
False
Call function
avg = get_average(arr)
Declare integer
count = 0
i = 0
i < len(arr)?
False
Return
count
True
arr[i] > avg?
True
Calculate
count = count + 1
i = i + 1
Calculate
i = i + 1
Return to main
program

```python
#d. Count number of elements with value higher than the average
def count_higher_than_average(arr):
    if len(arr) == 0:
        return 0
    avg = get_average(arr)
    count = 0
    i = 0
    while i < len(arr):
        if arr[i] > avg:
            count += 1
        i += 1
    return count
```

```
Array: [3, 8, 1, 14, 9, 22, 6]
Count of elements > average: 2
```

e. check whether values of an array are increasing (sorted)



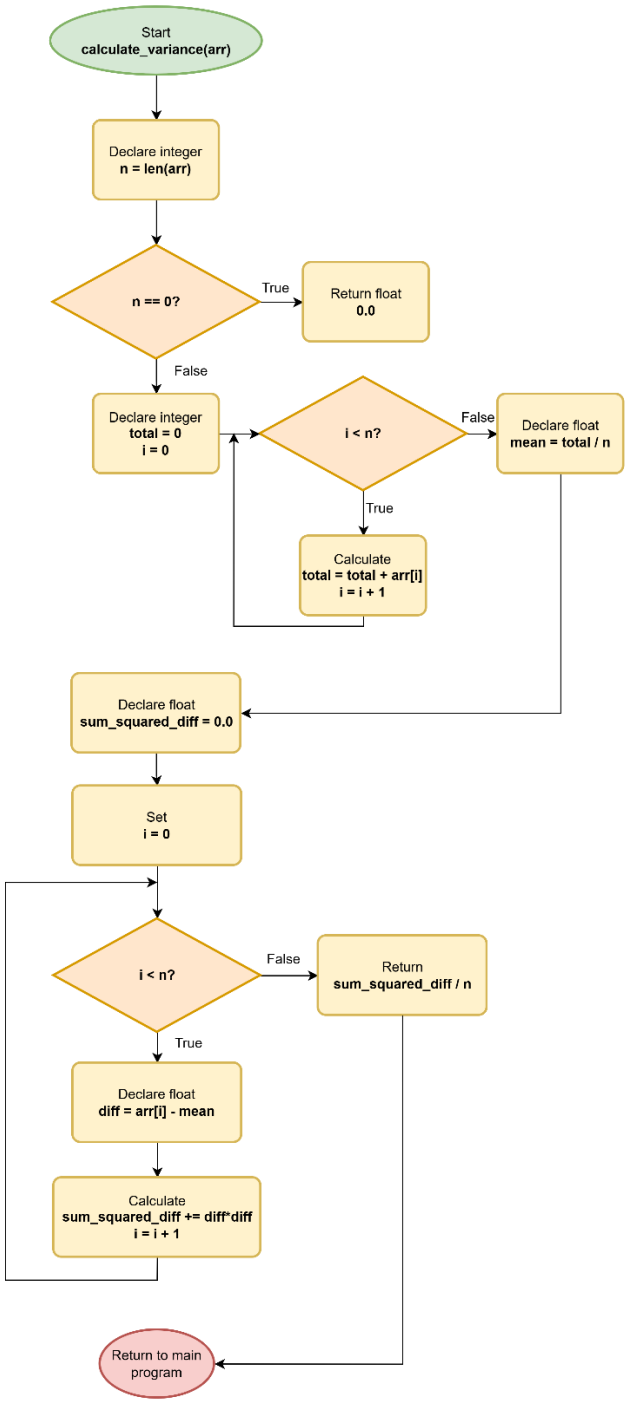
```python
#e. Check whether values of an array are increasing (sorted)
def is_increasing(arr):
    if len(arr) <= 1:
        return True
    i = 0
    while i < len(arr) - 1:
        if arr[i] > arr[i + 1]:
            return False
        i += 1
    return True
```

```
Array: [3, 8, 1, 14, 9, 22, 6]
Is increasing: False
Is [2, 5, 7, 10] increasing: True
```

ข้อที่ 2 ให้นักศึกษาเขียนฟังก์ชันที่รับพารามิเตอร์เป็นอะเรย์ของจำนวนเต็ม (integer)

และ return ค่าความแปรปรวน (Variance)

การคำนวณค่าความแปรปรวนมีสูตรคือ $\sigma^2 = \frac{\sum_{i=1}^{n}(x_i-\mu)^2}{n}$ โดยหาจากผลรวมของผลต่างกำลังสองระหว่าง ค่าแต่ละค่าของข้อมูลกับค่าเฉลี่ย และนำค่านั้นหารด้วยจำนวนข้อมูล



```python
def calculate_variance(arr):
    n = len(arr)
    if n == 0:
        return 0.0

    #Calculate the mean
    total = 0
    i = 0
    while i < n:
        total += arr[i]
        i += 1
    mean = total / n

    #Calculate the sum of squared differences
    sum_squared_diff = 0.0
    i = 0
    while i < n:
        diff = arr[i] - mean
        sum_squared_diff += diff * diff
        i += 1

    #Compute population variance
    return sum_squared_diff / n
```

```
Array: [10, 12, 23, 23, 16, 23, 21, 16]
Variance: 24.0
```